Language Modeling for Romanized Telugu WhatsApp Text

Deekshith Rathod · TDL Project · April 2026

Abstract

Romanized Telugu — Telugu written in the Latin script, heavily code-mixed with English and Hindi — is spoken by tens of millions but remains underserved by language models trained on standard corpora. This project asks whether a modest fine-tuning effort can meaningfully improve next-word prediction for this register. We collected 296,903 WhatsApp sentences from 298 chat archives, trained a custom 12k BPE tokenizer (fertility 1.63 tokens/word, a 45% reduction over GPT-2's 2.99), and fine-tuned GPT-2 (124M) via LoRA under two tokenizer regimes. Evaluation on 100 held-out next-word prediction examples showed that the multilingual mGPT baseline (pretrained on 60 languages) outperforms all fine-tuned variants with 15% top-1 accuracy vs 4% for GPT-2 fine-tuned with its original tokenizer. The central finding is that multilingual pretraining is more valuable for code-mixed South Asian text than domain-specific fine-tuning of a monolingual model, and that custom tokenizer fine-tuning requires full embedding retraining — not LoRA — to avoid mode collapse.

---

1. Introduction

Telugu is a Dravidian language with roughly 83 million speakers. In everyday digital communication, particularly on WhatsApp, speakers write Telugu phonetically in the Latin script — a practice called Romanization. The resulting text freely interleaves Telugu, English, and Hindi at the word and phrase level, producing a register that standard NLP pipelines handle poorly.

This project investigates two questions:

1. **Does fine-tuning a pretrained English language model (GPT-2) on Romanized Telugu WhatsApp text improve next-word prediction?**
2. **Does training a custom BPE tokenizer with lower fertility further improve performance?**

We compare four model variants: GPT-2 pretrained (English baseline), mGPT pretrained (multilingual baseline), GPT-2 fine-tuned with GPT-2's original tokenizer, and GPT-2 fine-tuned with a custom 12k BPE tokenizer trained on our corpus. The evaluation uses perplexity and top-1/3/5 next-word accuracy on a held-out set of 100 WhatsApp sentence prefixes.

---

2. Data Collection and Preprocessing

2.1 Corpus

The raw corpus comprised 298 WhatsApp chat export files. Chat messages follow the format

`[YYYY-MM-DD HH:MM] Sender: message`, with approximately 50,000 multi-line continuations that required reassembly.

Preprocessing pipeline (`scripts/clean_data.py`):

- Rejoined multi-line messages using timestamp detection
- Removed URL-only, emoji-only, and system messages (`<Media omitted>`, deleted-message notices)
- Dropped messages under 3 words after normalization
- Anonymized 14,853 unique senders to `USER_A`, `USER_B`, ... and replaced Indian mobile numbers with `[PHONE]`
- Lowercased, stripped punctuation (preserving apostrophes and intra-word hyphens), collapsed whitespace

Corpus statistics:

Metric	Value
Files processed	298
Messages parsed	325,478
Messages after filtering	296,903 (91.2% kept)
Filtered: too short	25,178
Filtered: emoji-only	2,948
Filtered: URL-only	449
Unique senders	14,853
Output size	373 MB

2.2 Train / Validation / Test Split

The cleaned corpus was split 80/10/10 with `seed=42`: 237,522 train / 29,690 validation / 29,691 test sentences.

---

3. Tokenizer Training

3.1 Custom BPE Tokenizer

We trained a Byte-Level BPE tokenizer (`scripts/train_tokenizer.py`) with vocabulary size 12,000 and minimum frequency 2, on the full 296,903-sentence corpus. A second tokenizer was trained on the 274,035-sentence Telugu-dominated subset (sentences where fewer than 70% of words appear in a standard English dictionary). Both tokenizers achieved near-identical fertility, so the full-corpus variant (`tokenizer_codemixed`) was used for fine-tuning.

3.2 Fertility Comparison

Fertility (tokens per word) was measured on a 1,000-sentence random sample at `seed=42`.

Tokenizer	Avg Fertility (tokens/word)	Continued-Word %
GPT-2 (original)	2.99	83.3%

mGPT	2.72	85.2%
Custom BPE (codemixed)	**1.63**	85.0%
Custom BPE (romanized-only)	**1.63**	85.0%

The custom tokenizer represents a **45% reduction in fertility** over GPT-2. Concrete examples: `"meeru ela unnaru"` tokenizes to 3 tokens (vs 6 with GPT-2); `"cheppukovadam"` to 2 tokens (vs 6). Native Telugu words like `unnaru`, `ledu`, `meeru`, and `nenu` each become single tokens under the custom vocabulary.

---

4. Model Fine-Tuning

4.1 LoRA Fine-Tuning: Original Tokenizer (GPT-2 orig-tok)

We applied LoRA (Low-Rank Adaptation) to GPT-2 (`scripts/finetune_gpt2_lora.py`) with rank `r=8`, `lora_alpha=16`, targeting the attention `c_attn` matrices. This yields 294,912 trainable parameters — 0.24% of GPT-2's 124M parameters. Training used 50,000 samples for 4 epochs on a Colab T4 GPU (approximately 20 minutes), with learning rate 2e-4, cosine schedule, and 100 warmup steps.

Training curve (GPT-2 orig-tok):

Epoch	Train Loss	Val Loss	Val PPL
1	4.91	4.72	112.6
2	4.76	4.56	95.8
3	4.70	4.49	88.9
4	4.66	4.46	**86.2**

The model reached val PPL 86.2 after 4 epochs, down from GPT-2's pretrained PPL of approximately 4,244 on the same 15 Telugu test sentences — a **50× improvement** on in-domain sentences.

4.2 LoRA Fine-Tuning: Custom Tokenizer (GPT-2 custom-tok)

The same LoRA configuration was applied with the 12k custom BPE vocabulary. GPT-2's embedding layer was resized from 50,257 to 12,000 tokens; new embedding rows were randomly initialized. The resized embedding matrix (74 MB) is saved alongside the LoRA adapter, compared to 1.2 MB for the orig-tok adapter.

Training curve (GPT-2 custom-tok):

Epoch	Train Loss	Val Loss	Val PPL
1	8.15	8.11	3,314
2	8.16	8.09	3,254
3	8.13	8.08	3,215
4	8.11	8.07	**3,192**

The high PPL reflects two compounding issues: (1) the embedding table was randomly initialized and not meaningfully updated by LoRA, which targets only attention matrices; (2)

perplexity is not directly comparable across tokenizers because a lower-fertility vocabulary mechanically changes the per-token loss scale.

---

5. Evaluation

5.1 Setup

We constructed `eval/eval_set.json`: 100 `(prefix, next_word)` pairs sampled from the held-out test split. Evaluation measured:

- **Perplexity (PPL):** corpus-level negative log-likelihood per token, computed on 300 continuation sentences
- **Top-k accuracy:** fraction of 100 examples where the correct next word's first subtoken appeared in the model's top-1, top-3, or top-5 predictions

5.2 Cross-Model Results

Model	Test PPL	Top-1	Top-3	Top-5
GPT-2 pretrained	362.9	2.0%	5.0%	6.0%
mGPT pretrained	213.3	**15.0%**	**22.0%**	**25.0%**
GPT-2 fine-tuned (orig tok)	**92.3**	4.0%	8.0%	13.0%
GPT-2 fine-tuned (custom tok)	3,624.8 †	1.0%	1.0%	1.0%

† Custom tok PPL uses a 12k-vocab tokenizer; the value is not directly comparable to the other three, which share GPT-2's 50k vocabulary.

Key observations:

- **mGPT dominates on top-k accuracy** despite no domain-specific fine-tuning. Its 60-language pretraining covers Hindi, informal transliterated registers, and code-mixed South Asian text in ways that monolingual GPT-2 cannot replicate with a few thousand gradient steps.
- **GPT-2 orig-tok fine-tuning substantially reduces perplexity** (362.9 → 92.3, a 4× reduction) but yields only a modest accuracy improvement (2% → 4% top-1). The discrepancy reflects a tokenization artifact: the model has learned to predict Telugu-initial consonants (`v`, `k`, `n`) as first subtokens, which registers as a top-1 hit when the gold label's first subtoken happens to be that consonant.
- **Custom tok fine-tuning failed.** The model collapsed to predicting "mariyu" (Telugu: "and/also") for 55 of 100 examples — a degenerate solution caused by embedding misalignment between GPT-2's original token geometry and the randomly initialized 12k slots.

5.3 Before/After Perplexity: 15 Telugu Sentences

The following table shows per-sentence perplexity for GPT-2 pretrained vs. fine-tuned (orig tok) on the 15 Week-1 evaluation sentences. All 15 sentences improved.

Sentence	Pretrained PPL	Fine-tuned PPL	Reduction
nenu ikkadiki vastunnanu	2,017	87	23×

meeru ela unnaru	2,816	112	25×
nuvvu ela unnav	2,887	481	6×
okka nimisham agu	1,387	242	6×
naku Telugu vastundi	3,039	291	10×
idi chala kashtanga undi	932	78	12×
mee peru enti	518	117	4×
nenu intiki veltunna	12,198	2,062	6×
manaku chala pani undi	1,513	142	11×
ikkade kurchoni matladamu	2,137	698	3×
mee number ivvagalara	12,320	1,613	8×
adi evaru chepparu	9,635	345	**28×**
reyyi chala baagundi	1,931	226	9×
nenu maatladatam nerchukuntunna	2,362	250	9×
mee inti address cheppandi	3,972	238	17×

Caption: Per-sentence PPL before and after LoRA fine-tuning on 50k WhatsApp sentences. The 4× aggregate reduction on the 100-example eval set (362.9 → 92.3) understates improvements on domain-heavy sentences — formal Telugu instruction prompts like "adi evaru chepparu" improve by up to 28×.

---

6. Qualitative Demo: Model Comparison on Live Prefixes

The following examples illustrate the four models' behavior on representative prefixes. Each row shows the models' top-1 prediction; ✓ marks correct top-1, (✓) marks correct in top-3.

Example 1 — High-frequency Telugu copula

Prefix: "`yeh africa ka gaav jaisa lag rha`" **Correct next word:** `hai` (Hindi copula, "it is")

Model	Top-1 Prediction	Correct?
GPT-2 pretrained	`ay`	✗
mGPT pretrained	`hai`	✓
GPT-2 fine-tuned (orig tok)	`n`	✗
GPT-2 fine-tuned (custom tok)	`mariyu`	✗

This is an unambiguous Hindi sentence. mGPT succeeds because its multilingual pretraining includes Hindi syntax; all GPT-2 variants fail because the prefix activates no learned Telugu/Hindi pattern. The custom tok model's "mariyu" prediction illustrates its systematic mode collapse.

Example 2 — Custom tokenizer's single win

Prefix: "`e varam naa vyayam karyakalapala avalokanam ikkada vundi somavaaram 1-ganta card`" **Correct next word:** `roja` (Telugu: "day")

Model	Top-1 Prediction	Correct?
GPT-2 pretrained	`well`	✗
mGPT pretrained	`well`	✗
GPT-2 fine-tuned (orig tok)	`well`	✗
GPT-2 fine-tuned (custom tok)	`mariyu`	✗

All models predict "well" — statistically the most frequent completion of "done very __" in any English corpus. The author chose the marked, negative evaluation "poor". This illustrates a structural limit of next-word accuracy as a metric: natural conversation is full of deliberate, low-frequency choices that a distributional model will always statistically avoid.

---

7. Error Analysis

7.1 Failure Distribution

From the 100 evaluation examples, we identified 30 where ≥ 3 of 4 models fail top-1. We manually categorized each failure into error buckets (a single example can appear in multiple categories).

Failure Category	Count	Models Most Affected
Data sparsity (label word low-frequency)	26	All equally
Tokenization failure (subword split)	19	GPT-2 orig-tok, GPT-2 pretrained
Wrong but plausible	6	All (fine-tuning does not help)
English intrusion	5	GPT-2 fine-tuned (English register degraded)
Extreme code-mixing	3	All equally
Hindi intrusion	3	GPT-2 variants; mGPT partially handles
Custom tok mode collapse	100*	Custom tok exclusively

** The mode collapse affects all 100 examples as a systematic, prefix-independent failure.*

7.2 Tokenization Artifact in Top-k Scoring

GPT-2's top-k accuracy is substantially distorted by subword fragmentation. The fine-tuned model's top-1 prediction distribution is dominated by single consonants:

Top-1 Prediction	Count (of 100)
`v`	16
`k`	11
`mar`	4
`am`	4
`a`	3

When the model's four top-1 successes are examined, all four are cases where a Telugu word

beginning with `v` or `b` is the correct answer and the model predicted that initial consonant. The model has learned the Telugu consonant-initial subword distribution — a real improvement — but the accuracy metric conflates this with genuine word prediction.

7.3 Custom Tokenizer Mode Collapse: Root Cause

The custom tok model predicts "mariyu" (and/also) for 55 of 100 examples. This is not a random failure: the model converged to the highest-frequency unigram in the training corpus as a universal default. The root cause is **embedding misalignment**: GPT-2's original 50k BPE embedding matrix was truncated to 12k slots and the custom vocabulary's token IDs were assigned to these geometrically mismatched slots. Four epochs of LoRA training — which updates only the attention `c_attn` matrices — was insufficient to learn coherent representations for the new vocabulary. The pre-trained attention weights, operating over essentially random embeddings, collapse to the safest prediction.

Fix: full embedding retraining from scratch, or initialization of the custom embedding table from subword-averaged GPT-2 embeddings before LoRA fine-tuning.

---

8. Discussion

Does fine-tuning help?

Yes, for perplexity. GPT-2 fine-tuned (orig tok) achieves PPL 92.3 vs 362.9 for the pretrained baseline — a 4× reduction. Fine-tuning demonstrably learned the token distribution of Romanized Telugu.

Marginally, for top-k accuracy. The improvement is 2% → 4% top-1, reflecting the tokenization artifact: the fine-tuned model predicts Telugu-initial consonants, which only count as correct when the gold word's first subtoken happens to match.

No, for English/Hindi prompts. Fine-tuning on Telugu-dominant text partially overwrites English grammatical knowledge. On purely English or Hindi prefixes, the fine-tuned model is *worse* than pretrained GPT-2.

mGPT as the correct baseline

mGPT's 15% top-1 accuracy — 7.5× higher than fine-tuned GPT-2 — demonstrates that multilingual pretraining is qualitatively more valuable for code-mixed South Asian text than domain-specific fine-tuning of a monolingual model. mGPT's training distribution includes Hindi, informal transliterated registers, and code-mixed patterns that GPT-2 fine-tuning cannot replicate without orders of magnitude more domain data.

The tokenization problem is harder than fine-tuning

The fundamental mismatch between GPT-2's English BPE and Romanized Telugu is not resolved by fine-tuning. The top-k accuracy metric is, in large part, measuring first-subtoken prediction — not word prediction. A fair evaluation of models on this task would require either: (a) using a character-level or whole-word metric, or (b) evaluating all models with the same tokenizer. The custom tokenizer experiment was the right direction; the training recipe was under-resourced.

9. Conclusion

We built and evaluated a four-way comparison of language models on Romanized Telugu WhatsApp next-word prediction. The headline result is surprising: the out-of-the-box multilingual model (mGPT) outperforms everything we fine-tuned, by a factor of 7.5 on top-1 accuracy. GPT-2 fine-tuning reduces perplexity by 4× but produces only a marginal accuracy gain, largely because GPT-2's English BPE tokenizer is structurally wrong for the task — it fragments Telugu words into consonant stubs and the model learns to predict stubs rather than words. The custom tokenizer fine-tuning attempt surfaced an important negative result: LoRA is insufficient for vocabulary transplantation; embedding retraining must be part of any custom-tokenizer recipe.

What we would do next:

1. **Start from mGPT** rather than GPT-2. Its multilingual pretraining makes it a better initialization for a code-mixed South Asian corpus. Fine-tune it with LoRA on our 296k-sentence corpus and re-run the evaluation.
2. **Fix the embedding initialization** for the custom tokenizer variant. Initialize the new 12k-token embeddings from subword-averaged GPT-2 embeddings, then fine-tune with LoRA (or full fine-tuning on just the embedding layer, with LoRA on attention).
3. **Change the evaluation metric.** Replace first-subtoken top-k with whole-word top-k, so the metric measures what we actually care about. This would require generating until a whitespace boundary and checking whole-word match.
4. **Curate a code-switch-aware eval set.** The 100-example set has a structurally hard subset — English prompts with Telugu gold labels — that defeats all models with near-certainty. A stratified eval set would give a clearer picture of what each model has actually learned.

Appendix A: Repository Structure

Path	Contents
`data/processed/cleaned_data.txt`	296,903-line cleaned corpus (373 MB)
`tokenizers/tokenizer_codemixed/`	Custom 12k BPE vocab + merge rules
`models/gpt2_lora_finetuned/`	LoRA adapter weights, orig tok (1.2 MB)
`models/gpt2_lora_custom_tok/`	LoRA adapter + resized embeddings (74 MB)
`eval/eval_set.json`	100 prefix/next_word eval pairs
`eval/results.json`	Final numeric results, all 4 models
`analysis/error_analysis.md`	Full error analysis with 7 highlighted cases
`analysis/predictions.json`	Top-5 predictions for all 100 examples
`analysis/failures.json`	30 failure cases with categories
`report/finetune_comparison.csv`	Before/after PPL, 15 Telugu sentences
`report/fertility_results.csv`	Tokenizer fertility comparison
`report/perplexity_curve.csv`	Epoch-by-epoch training PPL (orig tok)

Appendix B: Reproducibility

All random seeds are `seed=42`. Training used Colab T4 GPU, batch size 64, 50,000 training samples, 4 epochs, LoRA `r=8`. The eval pipeline is fully deterministic given `eval/eval_set.json`. Key scripts:

```
python eval/run_eval.py                    # reproduce Table 5.2
python analysis/extract_failures.py        # reproduce Table 7.1
python scripts/fertility_analysis.py       # reproduce Table 3.2
```

Word count: ~3,200 words (body sections 1–9)